

Experiment 8

AIM: Use Postman to test sample API

1. Objective

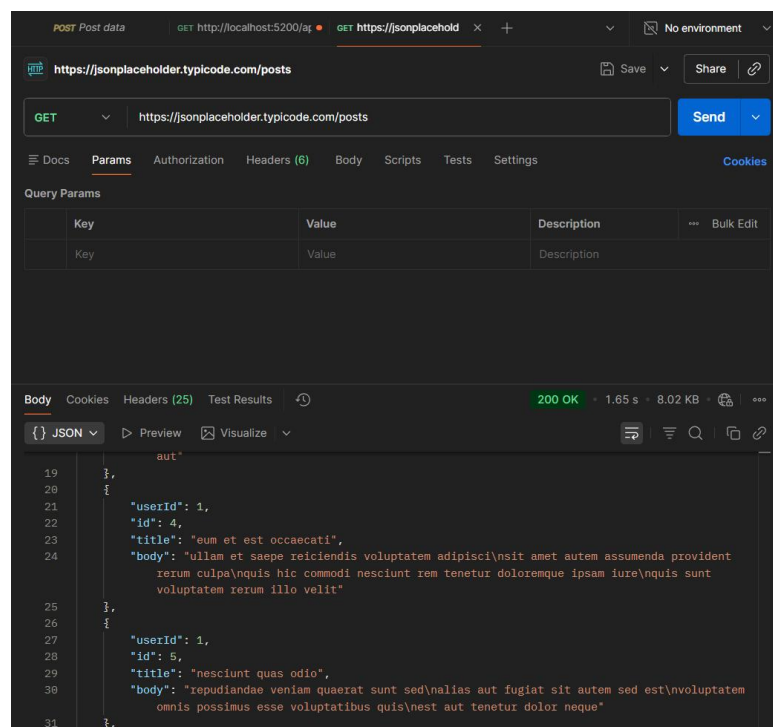
- Understand how to test REST APIs using Postman by sending different types of HTTP requests.
- Get
- Post
- Put
- Delete

2. Prerequisites

- postman
- API to be tested
 - Create a simple node.js API / use some free fake API

3. Test API

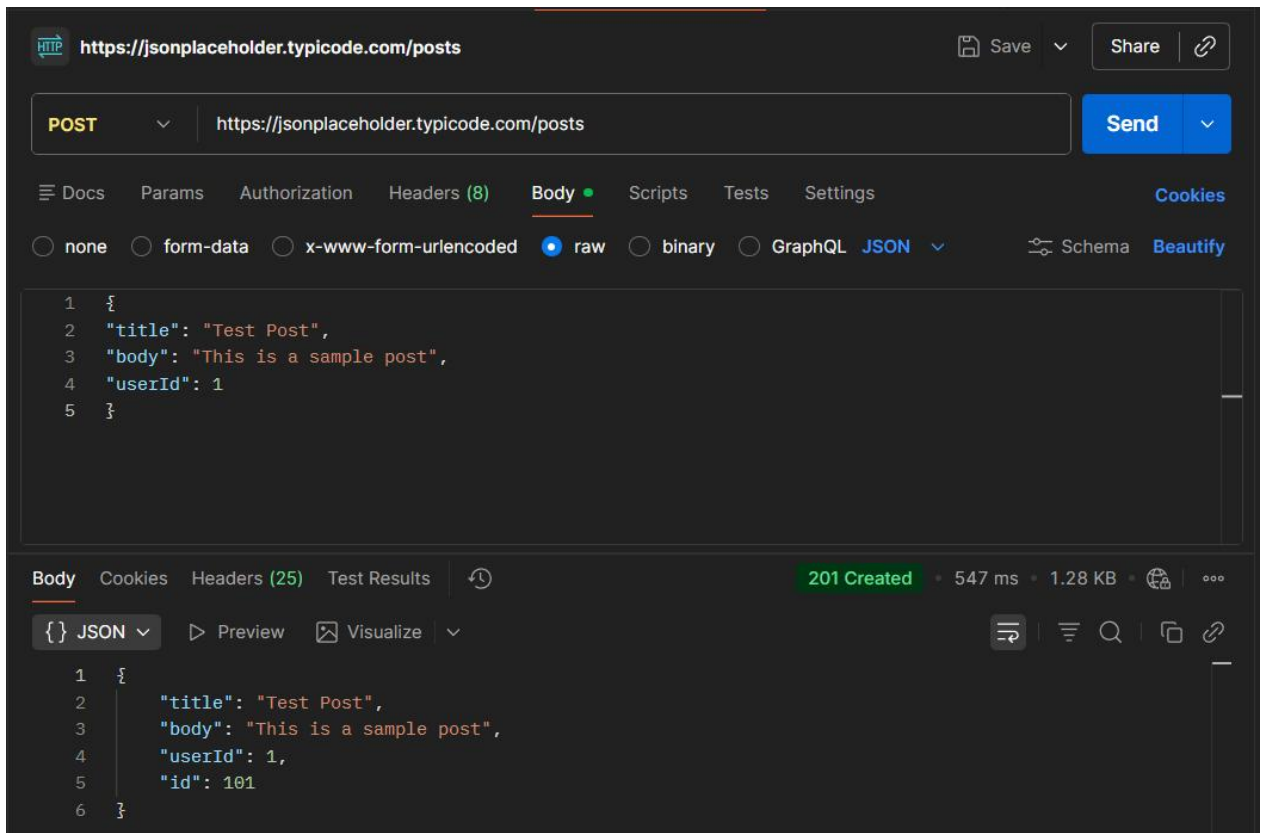
Get: URL: <https://jsonplaceholder.typicode.com/posts>



Post:

<https://jsonplaceholder.typicode.com/posts>

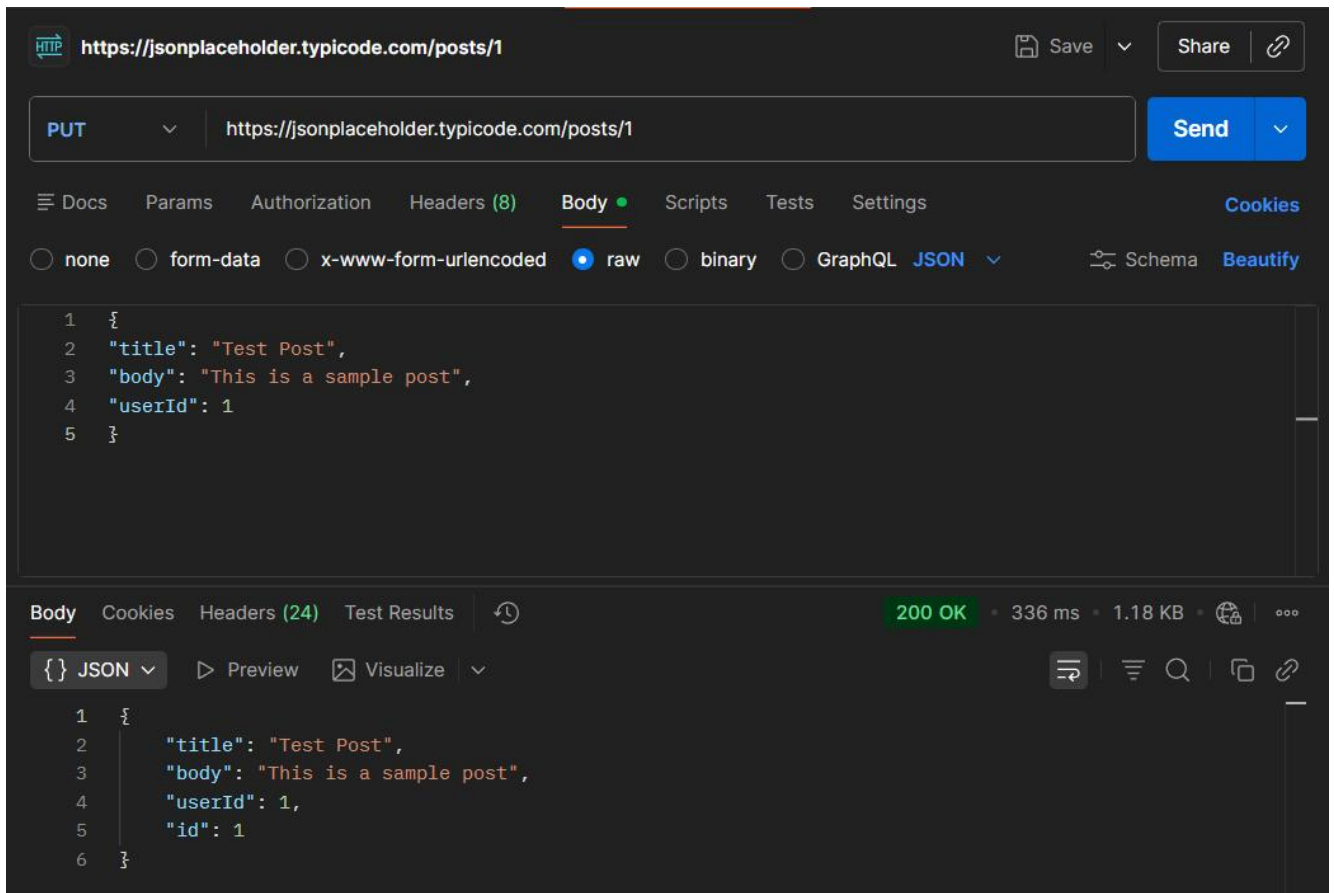
```
{  
  "title": "Test Post",  
  "body": "This is a sample post",  
  "userId": 1  
}
```



Put:

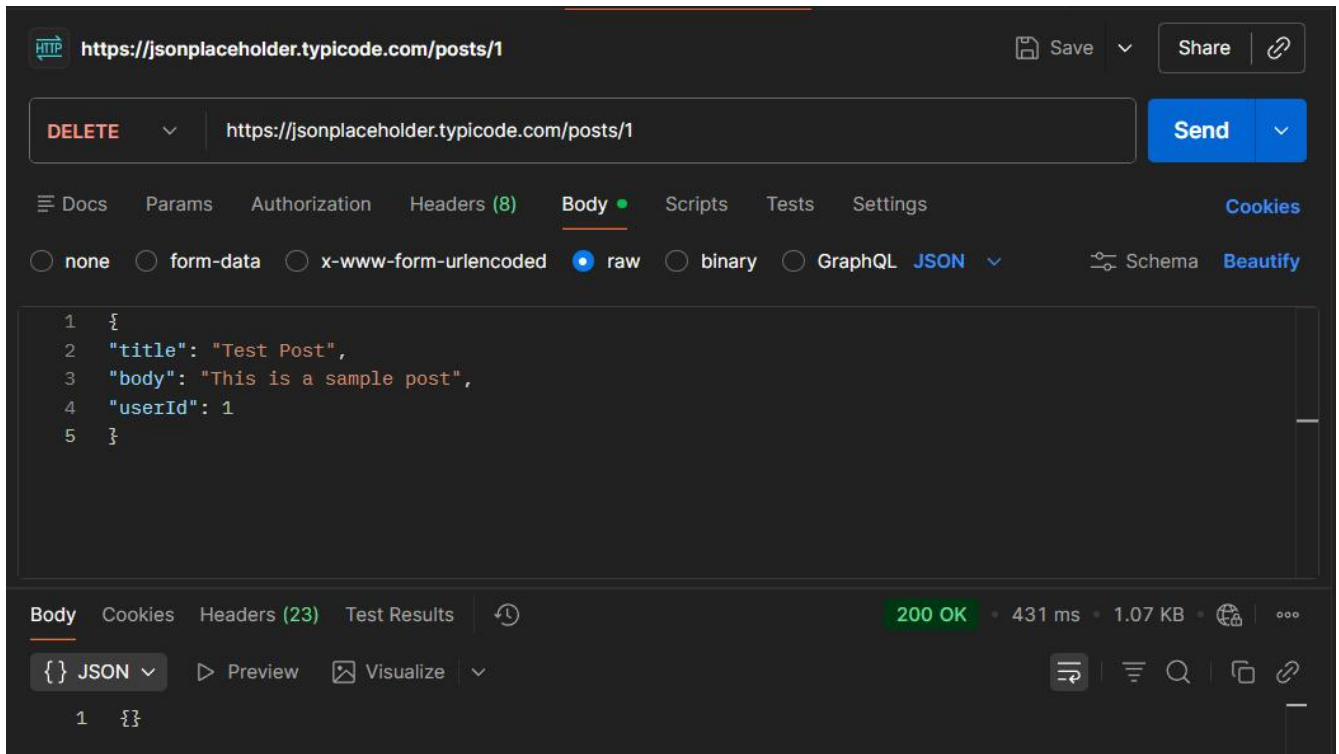
<https://jsonplaceholder.typicode.com/posts/1>

```
{  
  "id": 1,  
  "title": "Updated Title",  
  "body": "Updated content",  
  "userId": 1  
}
```



Delete:

<https://jsonplaceholder.typicode.com/posts/1>



4. Create node.js API

Step 1: Create Project

mkdir simple-api

cd simple-api

npm init -y

npm install express

Step 2: Create server.js file

```
const express = require('express');
```

```
const app = express();
```

```
app.use(express.json());
```

```
let students = [
```

```
  { id: 1, name: "Kishan", branch: "CE" },
```

```
  { id: 2, name: "Tarak", branch: "CE" }
```



```
];  
app.get('/students', (req, res) => {  
    res.json(students);  
});  
app.get('/students/:id', (req, res) => {  
    const student = students.find(s => s.id == req.params.id);  
    res.json(student);  
});  
app.post('/students', (req, res) => {  
    const newStudent = {  
        id: students.length + 1,  
        name: req.body.name,  
        branch: req.body.branch  
    };  
    students.push(newStudent);  
    res.status(201).json(newStudent);  
});  
app.put('/students/:id', (req, res) => {  
    const student = students.find(s => s.id == req.params.id);  
    if (student) {  
        student.name = req.body.name;  
        student.branch = req.body.branch;  
        res.json(student);  
    } else {  
        res.status(404).json({ message: "Student not found" });  
    }  
});  
  
// DELETE - Remove student
```



```
app.delete('/:id', (req, res) => {  
    students = students.filter(s => s.id !== req.params.id);  
    res.json({ message: "Student deleted" });  
});  
app.listen(3000, () => {  
    console.log("Server running on http://localhost:3000");  
});
```

Step 3: Run server

node server.js

```
C:\Windows\System32\cmd.e  X  +  v

Microsoft Windows [Version 10.0.26200.8039]
(c) Microsoft Corporation. All rights reserved.

C:\Users\kisha>mkdir simple-api

C:\Users\kisha>cd simple-api

C:\Users\kisha\simple-api>npm init -y
Wrote to C:\Users\kisha\simple-api\package.json:

{
  "name": "simple-api",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "description": ""
}

C:\Users\kisha\simple-api>npm install express

added 65 packages, and audited 66 packages in 4s

22 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

C:\Users\kisha\simple-api>notepad server.js

C:\Users\kisha\simple-api>node server.js
Server running on http://localhost:3000
|
```

5. Procedure TEST using postman

1. GetAll:

Type: Get

URL : <http://localhost:3000/students>

Check response



The screenshot shows a REST client interface with the following details:

- URL:** `http://localhost:3000/students`
- Method:** GET
- Response Status:** 200 OK
- Response Time:** 48 ms
- Response Size:** 313 B
- Response Body (JSON):**

```
[
  {
    "id": 1,
    "name": "Kishan",
    "branch": "CE"
  },
  {
    "id": 2,
    "name": "Tarak",
    "branch": "CE"
  }
]
```

2. Post:

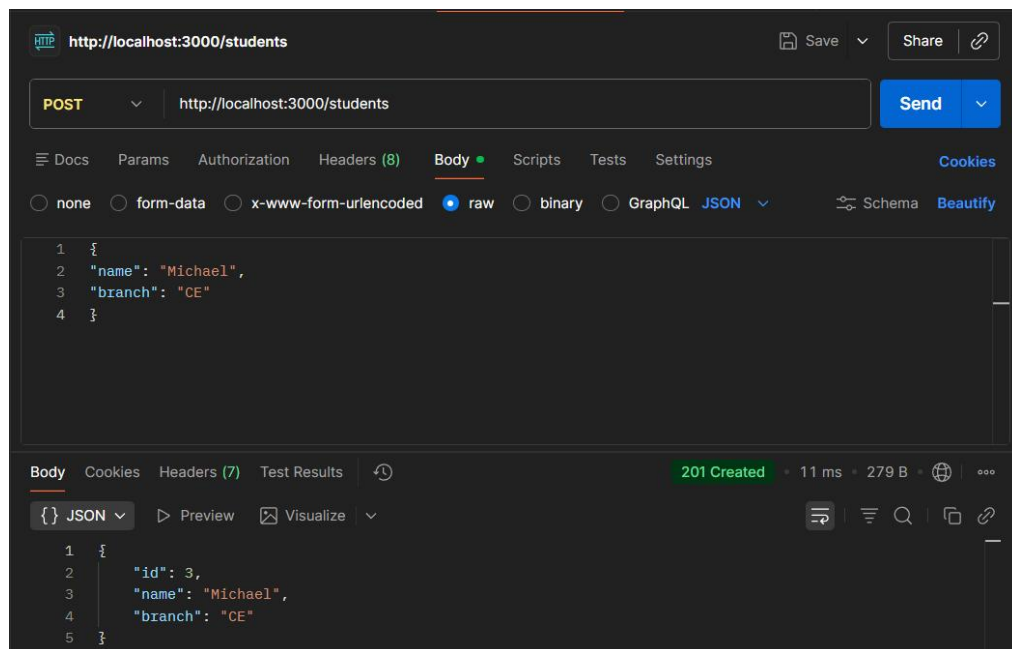
Type: Post

URL: `http://localhost:3000/students`

Body:

```
{
  "name": "Michael",
  "branch": "CE"
}
```

Check response and Send Get request to see new data inserted



3. Put request:

Type: PUT

URL : `http://localhost:3000/students/1`

Body:

```
{  
  "name": "Kishan Vachhani",  
  "branch": "IT"  
}
```

Check response

Send Get request to see updated data



PUT http://localhost:3000/students/1

Body

```
1 {
2   "name": "Kishan Vachhani",
3   "branch": "IT"
4 }
```

200 OK • 15 ms • 282 B

Body

```
1 {
2   "id": 1,
3   "name": "Kishan Vachhani",
4   "branch": "IT"
5 }
```

4. Delete request:

Type: DELETE

URL : http://localhost:3000/students/1

Check response

Send Get request to see updated data

DELETE http://localhost:3000/students/1

Body

```
1 {
2   "name": "Kishan Vachhani",
3   "branch": "IT"
4 }
```

200 OK • 12 ms • 264 B

Body

```
1 {
2   "message": "Student deleted"
3 }
```